



Predicate接口与对象筛查

北京理工大学计算机学院
金旭亮

示例用到的数据类与测试集合

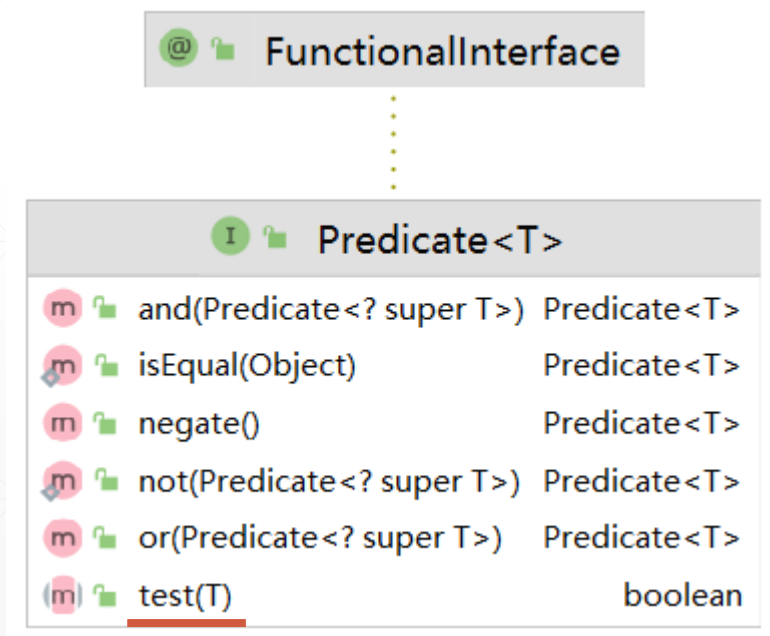
Person		
f	age	int
f	name	String
m	getAge()	int
m	getName()	String
m	setAge(int)	void
m	setName(String)	void
m	toString()	String

//创建用于测试的Person对象集合

```
static List<Person> createPersonCollection() {  
    List<Person> people = new ArrayList<>();  
    people.add(new Person( name: "张三", age: 48));  
    people.add(new Person( name: "李四", age: 30));  
    people.add(new Person( name: "王五", age: 73));  
    return people;  
}
```

Predicate<T>接口

Predicate<T>接口主要用于定义一个“判断规则”，其中最重要的方法是test()，它接收一个T类型的对象，然后判断这个对象是否满足某个条件，返回true/false值。



这一接口多用于筛选数据。

Predicate<T>接口示例

将Predicate<T>用于为集合对象指定一个筛选条件：

```
//最原始的Java实现方式
static void traditionalCode(List<Person> people) {
    var predicate = new Predicate<Person>() {
        @Override
        public boolean test(Person person) {
            return (person.getAge() > 60);
        }
    };

    for (Person person : people) {
        if (predicate.test(person)) {
            System.out.println(person);
        }
    }
}
```

只选出60岁以上的人，可以进一步简化为：



```
Predicate<Person> predicate = p->p.getAge() > 60;
```

使用Lambda表达式进行简化

使用Lambda表达式，前面的代码可以简化为：

```
//使用Lambda表达式的实现方式
static void useLambdaCode(List<Person> people) {
    Predicate<Person> oldPerson = p -> p.getAge() > 60;
    people.forEach(p -> {
        if (oldPerson.test(p)) {
            System.out.println(p);
        }
    });
}
```

推荐的写法

//应用模块化与函数式编程思想消除重复代码

```
static void simplestCode(List<Person> people) {  
    //动态指定数据筛选条件  
    Predicate<Person> predicate = p -> p.getAge() > 60;  
    Predicate<Person> predicate2 = p -> p.getAge() < 60;  
  
    //显示老年人  
    displayPeople(people, predicate);  
    //显示年青人  
    displayPeople(people, predicate2);  
}
```

//将筛选条件作为方法参数传入

```
private static void displayPeople(List<Person> people,  
                                   Predicate<Person> predicate) {  
    people.forEach(p -> {  
        if (predicate.test(p)) {  
            System.out.println(p);  
        }  
    });  
}
```

重构代码，使用Lambda表达式
“动态”地指定筛选条件。

这种编码方式产生的代码可维护
性较强，推荐。

实例：PredicateLambda.java

级联的Predicate

@ FunctionalInterface

I Predicate<T>

- m and(Predicate<? super T>) Predicate<T>
- m isEqual(Object) Predicate<T>
- m negate() Predicate<T>
- m not(Predicate<? super T>) Predicate<T>
- m or(Predicate<? super T>) Predicate<T>
- m test(T) boolean

```
private static void LinkedPredicate() {  
    Predicate<Integer> p = i -> i % 2 == 0;  
    Predicate<Integer> q = i -> i % 3 == 0;  
    Predicate<Integer> r = i -> i > 10 & i < 40;  
    //生成[1, 50]区间的整数集合  
    var nums : List<Integer> = IntStream.rangeClosed(1, 50).boxed().toList();  
    //输出 (10, 40) 区间内2和3的倍数  
    nums.stream().filter(p.and(q).and(r)).forEach(System.out::println);  
}
```



上面的例子展示了and方法的使用，请编写示例，掌握not、or和negate这几个方法的使用。

级联使用Predicate时要注意

and和or方法是按照在表达式链中的位置，从左向右确定优先级的。因此，`a.or(b).and(c)`可以看作`(a || b) && c`。同样，`a.and(b).or(c)` 可以看作`(a && b) || c`。

JDK所提供的其他Predicate接口

BiPredicate: 接收两个参数

IntPredicate

LongPredicate

DoublePredicate

针对原始数据类型的增强版本



请同学们自行掌握以上接口的用法。